Batch 9 · Large-number multiplication in an HSM: analysis sheet

Choice: Karatsuba (3 half-size products, T(n) = 3T(n/2) + Θ(n) = Θ(n^1.585)), schoolbook base case at 16 limbs, carry kept as a separate bit so control flow and addresses depend only on n. Rejected alternative implemented and timed: schoolbook. Instance 5678 × 1234 = 7,006,652 verified by hand trace (slide 7) and by code (karatsuba_trace.py).

Table 1 · Method comparison for 4096-bit operands (n = 64 limbs of 64 bits)

Method	Time	Base-case limb mults at n = 64	Wins when (limbs)	Memory / constant-time notes	Status here
Schoolbook	Θ(n^2)	4,096 (measured: 3.53 µs)	below ~16	O(1) extra; trivially constant-time	implemented (rejected)
Karatsuba	Θ(n^1.585)	2,304 at T=16 · 1,728 at T=8 · 729 at T=1 (measured: 2.78 µs at T=16)	~16 up to a few hundred	~4n scratch = 2.5 KB; carry bit handled with masks	implemented (chosen)
Toom-Cook-3	Θ(n^1.465)	5 products of size n/3; 3 does not divide 64, so needs padding	hundreds to ~1,000	evaluation/interpolation divide by 3 (exact division, extra adds)	analysis only
FFT / Schönhage–Strassen	Θ(n log n log log n)	not meaningful below thousands of limbs	thousands	large transform buffers; many adds and twiddles	analysis only

Libraries switch at a threshold because the better exponent carries a larger constant (extra additions, evaluation, interpolation, memory traffic). Measured crossover here: Karatsuba ties schoolbook at 8–16 limbs and wins from 32; sweep of T at 32/64/256 limbs gives the best T = 16, matching the cut-over of 16 words in OpenSSL/BoringSSL.

Table 2 · Representation of a 4096-bit integer

Representation	Digit access	Add	Split by B^m	Space	Verdict
Array of 64-bit limbs, little-endian (chosen)	O(1)	64 steps	O(1) pointer views	512 B	one hardware MUL per pair; fixed width gives data-independent addresses
Decimal-digit array	O(1)	~1,233 steps	O(1)	~1.2 KB	~16× more digit operations (≈1.52 M for schoolbook)
Linked list of limbs	O(m)	O(m), pointer chasing	O(m) to find middle	~1 KB with pointers	cache-hostile; addresses depend on layout

Memoising sub-products: not inside one multiplication (ac, bd, (a+b)(c+d) are distinct at every level). Yes across signatures: per-key constants (Montgomery R^2, CRT values) and window tables are precomputed once and read by scanning every entry, so the lookup stays constant-time.

Table 3 · Measured running time (C, gcc -O2, 1 shared vCPU Xeon 2.1 GHz, best of 7)

Bits	Limbs	Schoolbook	Karatsuba (T=16)	Speed-up	Bits	Limbs	Schoolbook	Karatsuba (T=16)	Speed-up
512	8	0.06 µs	0.06 µs	0.97×	16,384	256	51.2 µs	27.0 µs	1.90×
1,024	16	0.20 µs	0.20 µs	1.02×	32,768	512	201 µs	84.3 µs	2.39×
2,048	32	0.96 µs	0.78 µs	1.23×	65,536	1,024	777 µs	262 µs	2.96×
4,096	64	3.53 µs	2.78 µs	1.27×	131,072	2,048	3.08 ms	0.79 ms	3.92×
8,192	128	13.2 µs	8.90 µs	1.48×	262,144	4,096	12.5 ms	2.30 ms	5.42×

Fitted exponent (≥ 64 limbs): schoolbook 1.965 vs derived 2; Karatsuba 1.618 vs derived $\log_2 3 = 1.585$. Ratios are meaningful; absolute times are not those of an HSM.

Table 4 · Worst case, constant-time and resource figures

Item	Result
Worst-case input	X = Y = 2^4096 − 1 (all-ones; 9999 × 9999 in decimal): every a+b, c+d carries at every node (carry census 2, 6, 18 on levels 0–2). Naive padded variant at 64 digits: 729 best, 4,767 random, 7,463 worst digit products vs 4,096 schoolbook. This code: 1,728 limb mults at T=8 for every input.
Timing leak and fix	Leaks: carry out of a+b / c+d (length trim), borrow in z1 − z2 − z0, zero-skipping. Fix: carry as a bit applied with masks, fixed-length add/sub, no early exit. Equal-timing check: random 2.75–2.79, all-ones 2.67, zero 2.51–2.60 µs; zero is 5–9% faster and unexplained, so a dudect-style test is still required.
Stack / scratch (n = 64)	Scratch 4n + 64 = 320 limbs = 2.5 KB; frames 160 B × 2–3 levels + 32 B (-fstack-usage); total under 4 KB of 512 KB (0.8%).
Throughput estimate (not measured on an HSM)	Assuming CRT, 5-bit windows, Montgomery reduction costed as a schoolbook product and my single-core timings: RSA-4096 ≈ 8.6 ms/signature → ≈117/s per core → ≈26 cores-equivalent for 3,000/s. RSA-2048 ≈ 1.0 ms → ≈1017/s per core → ≈3. The 4096-bit target therefore needs parallel modular-multiplier hardware; Karatsuba alone (≈20% saving) does not reach it.
Real-world reference	OpenSSL/BoringSSL bn_mul_recursive: Karatsuba, 16-word cut-over, 4·n2 scratch, middle term (a0 − a1)(b1 − b0) + a0b0 + a1b1 so no operand grows, versus our additive form with masked carries.